Observability

This directory contains Helm values for Prometheus/Grafana and Loki/Promtail, a PodMonitor for the worker, and an importable Grafana dashboard. It assumes the application is deployed in namespace `app` and uses `observability` for the monitoring stack.

Install

Install Helm and `kubectl`, connect to the target EKS cluster, and ensure the application is deployed first. Create the monitoring namespace and a Grafana admin Secret; supply a strong password through your normal secret-management process rather than committing it to this repository:

```
kubectl create namespace observability
kubectl create secret generic grafana-admin -n observability `
  --from-literal=admin-user=admin `
  --from-literal=admin-password='<set-a-strong-password>'
helm repo add prometheus-community https://prometheus-community.github.io/helm-
charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
helm upgrade --install monitoring prometheus-community/kube-prometheus-stack `
  --namespace observability `
  --values observability\prometheus-values.yaml
helm upgrade --install loki grafana/loki `
  --namespace observability `
  --values observability\loki\values.yaml
helm upgrade --install promtail grafana/promtail `
  --namespace observability `
  --values observability\loki\promtail-values.yaml
kubectl apply -f observability\prometheus\worker-podmonitor.yaml
```

The PodMonitor requires the Prometheus Operator CRDs installed by `kube-prometheus-stack`. It scrapes the worker's existing `/metrics` endpoint on the `metrics` port. Re-apply it after installing the monitoring stack if the CRD was not yet available.

Open Grafana by forwarding its service:

```
kubectl port-forward -n observability service/monitoring-grafana 3000:80
```

Log in with the username and password used to create `grafana-admin`, then import the autoscaling dashboard. Prometheus is installed as the default datasource; Loki is added by the values file. For service status and Prometheus targets, check:

```
kubectl get pods,pvc -n observability
kubectl port-forward -n observability service/monitoring-kube-prometheus-
```

```
prometheus 9090
```

The Prometheus UI is then available at `http://localhost:9090`. Confirm that the `worker-service` PodMonitor target is up before relying on dashboard data.

Metrics and limitations

The worker exports the following Prometheus metrics:

Metric	Type	Meaning
`worker_in_flight_jobs`	Gauge	Jobs currently being processed by this worker pod
`worker_jobs_processed_total`	Counter	Successfully processed jobs
`worker_jobs_duplicate_total`	Counter	Duplicate jobs skipped
`worker_jobs_error_total`	Counter	Failed jobs

Counters are process-local and reset when a pod restarts; use `rate()` or `increase()` and aggregate across pods for dashboards. Kubernetes replica panels use kube-state-metrics, included with kube-prometheus-stack.

The worker does not currently export SQS queue depth, per-job duration, or T0–T5 autoscaling timestamps. Queue depth is available from AWS CloudWatch and the KEDA scaler, but neither source is configured here as a Prometheus target. Consequently, the dashboard shows worker throughput, errors, in-flight work, replicas, and logs—not queue depth or control-loop latency. Do not interpret the scrape interval as an autoscaling latency measurement. The T0–T5 measurement process and its external evidence are described in `docs/control-loop-latency.md`.

Loki in this configuration is a single-binary, filesystem-backed deployment with one replica and seven-day log retention. It is suitable for development and controlled experiments, not highly available production use. Use durable, shared object storage and a production Loki deployment mode for production.